

CheckScan Desktop — Technical Architecture (Developer Guide)

Who this is for

This assumes general software development experience (APIs, databases, event-driven code) but **no prior exposure to scanner hardware or the TWAIN protocol**. Every scanning-specific concept is explained before it's used, so you can build this without having touched a scanner SDK before.

Glossary — scanning terms you'll need

Term	What it means
TWAIN	An industry-standard protocol (dating to the early 1990s, still dominant today) that lets software talk to scanners without writing driver code per model. Any scanner with a "TWAIN driver" can be controlled by any TWAIN-compatible app — including this one.
Data Source Manager (DSM)	A system-level component that acts as a switchboard between your app and whichever scanner driver is installed. Your app never talks to the scanner directly — it always goes through the DSM.
Data Source	TWAIN's term for "an installed scanner driver." Two scanners installed = two Data Sources. Your app asks the DSM "what sources exist?" and the user picks one.
ADF (Automatic Document Feeder)	The tray that lets you load a stack of paper and have the scanner pull sheets through one at a time, instead of scanning one page on a flatbed at a time. This is what makes batch check scanning practical.
Capability	TWAIN's configuration system. Before scanning, you "negotiate capabilities" — reading/writing named constants like <code>CAP_FEEDERENABLED</code> — to set things like resolution, color mode, or "use the feeder."
Native transfer	One of three ways TWAIN can hand you a scanned image: it gives you an in-memory bitmap directly. (The alternatives — memory transfer and file transfer — matter for very large or specialized workflows, not check-sized images.)

Term	What it means
DPI (dots per inch)	Scan resolution. Higher DPI = more detail and bigger files. 200–300 DPI is enough to keep a check’s printed text and MICR line (the magnetic ink numbers at the bottom) legible, without bloating file sizes.
NTwain	The open-source .NET library this project uses to talk to TWAIN, so you’re not hand-writing low-level Windows interop yourself.

Why this needs an unusual architecture

Two things make this different from a typical CRUD API, and they’re worth understanding before looking at any code:

1. TWAIN assumes a UI thread that doesn’t exist in a Web API. TWAIN was designed when every Windows app had exactly one UI thread pumping window messages, and its communication model depends on that message loop existing. A plain ASP.NET Core Web API is headless — no window, and request handling runs on thread-pool threads, not one dedicated UI thread. Section “The hidden window trick” below covers the fix.

2. A scan is a stream of events, not a single request/response. Triggering a scan doesn’t hand you back an image — it starts the driver working through the feeder, firing one event per page. Your code has to sit and listen for those events rather than `await`-ing a single call and getting a result.

Everything below is built around solving those two problems cleanly, once, so you never have to think about them again after the initial setup.

Stack and platform

Platform: Windows only

Scanner protocol: TWAIN only

UI shell: Electron

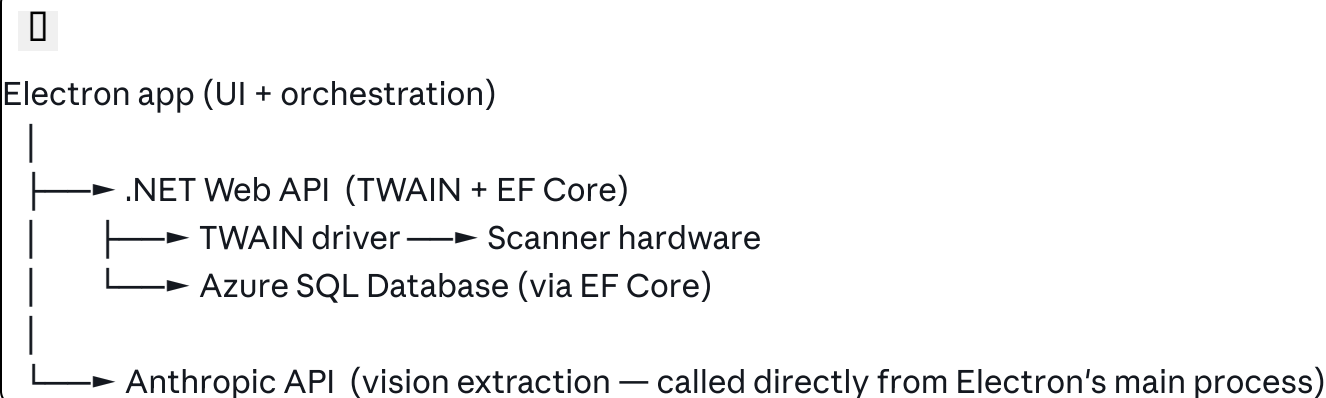
Scanner + data service: .NET minimal Web API (single project, top-level `Program.cs`, no MVC/controllers needed)

TWAIN library: NTwain

Data access: Entity Framework Core → Azure SQL Database

AI extraction: Anthropic API (Claude, vision-capable model), called from Electron's main process

How the pieces fit together



Electron's **renderer** (the actual browser-like window) never talks to the network directly. It goes through Electron's **main process**, which calls the local .NET API and the Anthropic API. This sidesteps CORS entirely and keeps the renderer sandboxed — standard Electron security practice.

The .NET API is spawned as a **child process** by Electron on launch, and killed on quit — it's not installed as a persistent Windows service, since it only needs to run while the app is open. It binds to `127.0.0.1` on a dynamically chosen free port, printing the port to stdout on startup; Electron's main process reads that line to know where to connect. **Never bind to `0.0.0.0`** — that would make it reachable from other machines on the network, which you don't want for something handling check images.

Published as a **self-contained single-file executable** (`dotnet publish -r win-x64 --self-contained -p:PublishSingleFile=true`), so the end user doesn't need the .NET runtime installed separately, and bundled inside Electron's resources folder.

The hidden window trick (making TWAIN work in a headless API)

Since a Web API has no window and no dedicated UI thread, you build one yourself on startup:

Spin up **one dedicated background thread** when the API starts.

On that thread, create a tiny **invisible native window** using Win32 interop (`CreateWindowEx` with no visible style — a handful of lines, not a real window anyone sees).

Run a **Windows message loop** on that same thread (`GetMessage`/`DispatchMessage` in a loop).

Give NTwain that hidden window's handle (HWND) as its TWAIN "parent."

From here, every TWAIN call gets marshaled onto that one thread — your actual API endpoints just post work onto it and `await` the result.

The HTTP layer doesn't need to know any of this is happening underneath.

This is a **known, solved pattern** — plenty of WinForms/WPF apps have done this exact trick to drive TWAIN from something that isn't really a UI app. You build it once, in one small class, and never touch it again.

```
using System;
using System.Threading;
using System.Threading.Tasks;

public class TwainThread
{
    private Thread _thread;
    private IntPtr _hiddenWindowHandle;

    public void Start()
    {
        _thread = new Thread(() =>
        {
            _hiddenWindowHandle = CreateInvisibleWindow(); // Win32 CreateWindowEx
            RunMessageLoop(); // GetMessage/DispatchMessage
        });
        _thread.SetApartmentState(ApartmentState.STA); // TWAIN requires an STA thread
        _thread.Start();
    }

    // Endpoints call into this to run TWAIN work on the correct thread
    public Task<T> RunOnTwainThread<T>(Func<T> work) { /* marshal + await */ }
}
```

Enumerating scanners

TWAIN sources are enumerated through the DSM. NTwain wraps this cleanly:

```
❏  
csharp  
// Inside a method running on the TWAIN thread  
using var session = new TwainSession(appId);  
session.Open();  
  
var sources = session.GetSources(); // list of installed TWAIN data sources  
  
var scannerList = sources.Select(s => new ScannerDto  
{  
    Id = s.Id,  
    Name = s.Name // friendly name, e.g. "Canon CR-190i"  
});
```

The API's `GET /scanners` endpoint just calls this on the TWAIN thread and returns the list as JSON. The Electron UI renders it as a dropdown; the user's choice is persisted (stored via EF Core, see below) as the default source for future scans.

Triggering a batch scan

This is the part that trips people up most, because it's not "call a function, get an image." It's: **configure → trigger → listen for events → stop when the feeder's empty.**

Step 1 — open the source and negotiate capabilities:

```
❏  
csharp
```

```

var source = session.OpenSource(selectedSourceId);

source.Capabilities.CapFeederEnabled.SetValue(true); // pull from the ADF, not the flatbed
source.Capabilities.CapAutoFeed.SetValue(true);    // auto-advance to the next sheet after each scan
source.Capabilities.ICapXferMech.SetValue(TransferMechanism.Native); // in-memory bitmap per page

bool hasCheckLoaded = source.Capabilities.CapFeederLoaded.GetCurrentValue();
if (!hasCheckLoaded)
    throw new ScannerException("Feeder is empty — load checks before scanning.");

```

Step 2 — trigger the scan:

```

[ ]
csharp
source.Enable(showUI: false, isModal: false, hiddenWindowHandle);
// This is the actual "go" signal. The driver physically picks up
// and scans the first check from the feeder.

```

Step 3 — listen for events as the feeder works through the stack:

```

[ ]
csharp

```

```

source.DataTransferred += (sender, e) =>
{
    Bitmap checkImage = e.NativeData; // one completed check, as an in-memory bitmap
    var jpegBytes = ConvertToJpeg(checkImage, dpi: 300);
    PushToElectron(jpegBytes); // e.g. via WebSocket, so the UI updates live
};

source.TransferReady += (sender, e) =>
{
    if (source.PendingXfers == 0)
    {
        source.Disable(); // feeder is empty — batch is done
    }
};

```

Because `CapAutoFeed` is on, you don't call anything again between pages — the driver keeps pulling sheets and firing `DataTransferred` automatically. Your job is just to collect (or stream) each image as it arrives, and clean up once `PendingXfers` reaches zero.

API endpoints

Endpoint	Purpose
<code>GET /scanners</code>	Enumerate installed TWAIN sources — friendly names + IDs
<code>POST /scanners/select</code>	Set the active/default scanner (persisted via EF Core)
<code>POST /scan</code>	Trigger a feeder batch scan; streams results as each check completes

Data layer: EF Core + Azure SQL

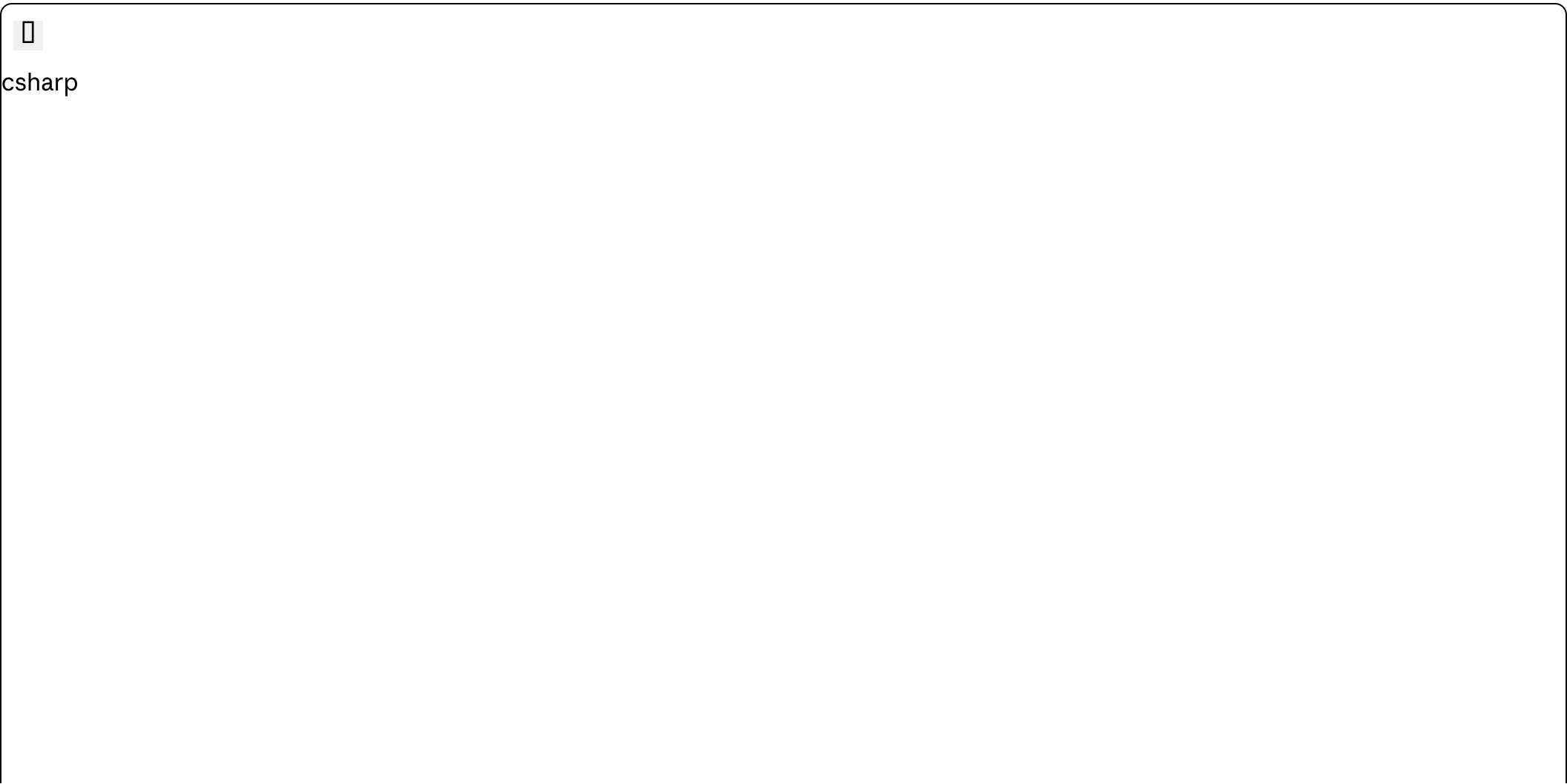
The same .NET project owns both the TWAIN layer and the data layer.

Packages:

`Microsoft.EntityFrameworkCore.SqlServer` — the Azure SQL / SQL Server provider (Azure SQL uses the same provider as on-prem SQL Server, just a different connection string)

`Microsoft.EntityFrameworkCore.Design` — needed at dev time for migrations

Entities:



```
public class Fundraiser
{
    public int Id { get; set; }
    public string Name { get; set; }
    public List<Check> Checks { get; set; } = new();
}
```

```
public class Batch
{
    public int Id { get; set; }
    public DateTime CreatedAt { get; set; }
    public string? Label { get; set; } // e.g. "Spring gala"
    public List<Check> Checks { get; set; } = new();
}
```

```
public class Check
{
    public int Id { get; set; }
    public int FundraiserId { get; set; }
    public Fundraiser Fundraiser { get; set; }
    public int BatchId { get; set; }
    public Batch Batch { get; set; }
    public decimal Amount { get; set; }
    public string Bank { get; set; }
    public DateTime CheckDate { get; set; }
    public string ImagePath { get; set; } // or a blob reference, see below
    public DateTime CreatedAt { get; set; }
}
```

DbContext:



```

csharp
public class CheckScanDbContext : DbContext
{
    public DbSet<Fundraiser> Fundraisers => Set<Fundraiser>();
    public DbSet<Batch> Batches => Set<Batch>();
    public DbSet<Check> Checks => Set<Check>();

    public CheckScanDbContext(DbContextOptions<CheckScanDbContext> options) : base(options) { }
}

// In Program.cs:
builder.Services.AddDbContext<CheckScanDbContext>(options =>
    options.UseSqlServer(connectionString));

```

Migrations: Use EF Core Migrations (`dotnet ef migrations add InitialCreate`, `dotnet ef database update`) to create and evolve the schema on Azure SQL. Call `db.Database.Migrate()` on API startup so the schema is always current without a manual step.

Connection string handling: This exe runs on a single personal machine you don't operate as a server, so don't hardcode the connection string or ship it in plain text config. Store it in **Windows Credential Manager** (a small wrapper, or the `CredentialManagement` package) or an encrypted local config file (via DPAPI / `ProtectedData.Protect`), and load it into memory only at API startup.

Firewall: Azure SQL only accepts allow-listed IPs by default. Since this runs from one machine, add that machine's IP to the Azure SQL server's firewall rules.

Image storage: Decide between check images as files on local disk (with just a path stored in the `Check` row — simpler, no extra Azure cost) or as blobs in Azure Blob Storage (fully cloud-backed, consistent with the Azure SQL choice, but an extra service to manage).

AI extraction: Anthropic API

Electron's main process sends each captured check image to Anthropic's Messages API (`https://api.anthropic.com/v1/messages`), using a vision-capable Claude model, requesting all seven fields the client doc commits to: **date, amount, bank, routing number, account number, check number, and donor/payer name.**

Request shape — the image goes as a base64-encoded content block alongside a text prompt:

```
❏  
  
javascript  
const response = await fetch("https://api.anthropic.com/v1/messages", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json",  
    "x-api-key": apiKey,  
    "anthropic-version": "2023-06-01"  
  },  
  body: JSON.stringify({  
    model: "claude-sonnet-4-6", // check docs.claude.com for the current recommended vision model  
    max_tokens: 1000,  
    messages: [{  
      role: "user",  
      content: [  
        { type: "image", source: { type: "base64", media_type: "image/jpeg", data: base64Image } },  
        { type: "text", text: EXTRACTION_PROMPT }  
      ]  
    }]  
  })  
});
```

Structured, per-field output with confidence: `EXTRACTION_PROMPT` should instruct Claude to respond with **only** JSON (no preamble, no markdown fences) matching a fixed shape — one object per field, each with a value and a confidence level:

```
❏  
  
json
```

```
{
  "date": { "value": "2026-08-12", "confidence": "high" },
  "amount": { "value": "250.00", "confidence": "high" },
  "bank": { "value": "Chase Bank", "confidence": "medium" },
  "routingNumber": { "value": "021000021", "confidence": "high" },
  "accountNumber": { "value": "1234567890", "confidence": "medium" },
  "checkNumber": { "value": "1042", "confidence": "high" },
  "donorName": { "value": "Maria Gomez", "confidence": "low" }
}
```

Parse this response, strip any markdown code fences defensively before `JSON.parse`, and pass it straight into the review screen — each field's confidence level drives the visual cue (e.g., low-confidence fields highlighted for extra scrutiny), matching the client-facing commitment that every field shows its confidence and stays editable.

Important caveat, worth knowing going in: this "confidence" is the model's own self-reported assessment when asked to rate itself, not a calibrated statistical probability the way a traditional OCR engine's confidence score is. It's a genuinely useful signal for flagging fields worth a closer look, but it shouldn't be treated as a precise accuracy percentage — which is exactly why the review step stays mandatory for every field regardless of the confidence shown.

API key handling: same principle as the Azure SQL connection string — don't hardcode it or ship it in plain text. Store it via Windows Credential Manager or an encrypted local config file, loaded into memory only when needed.

Start batch scan — Electron's main process calls `/scan`, referencing the saved default scanner.

Open source and configure feeder — API opens the TWAIN source and sets `CAP_FEEDERENABLED`, `CAP_AUTOFEED`, `ICapXferMech = Native`; checks `CAP_FEEDERLOADED` first to fail clearly if the feeder's empty.

Trigger the scan — `source.Enable(...)`. The driver picks up and scans the first check.

Capture each check — Each completed sheet fires `DataTransferred` with an in-memory bitmap. Autofeed keeps the driver advancing until `PendingXfers` hits 0, at which point the API calls `source.Disable()`.

Convert and stream results — Each bitmap is converted to JPEG (200–300 DPI) and pushed to Electron as it completes, rather than waiting for the whole batch.

Extract check data — Electron's main process sends each image to the Anthropic API, requesting date, amount, bank, routing number, account number, check number, and donor name, each with a confidence level.

Populate the review screen — Extracted data is added to the review list per check as it returns.

Confirm and save — User assigns a fundraiser and confirms; the record is written to Azure SQL via EF Core, linked to fundraiser, batch, and date.

Open technical decisions

Live vs. batch display — stream each check as it's scanned (better UX, needs WebSocket/SignalR), or return the whole batch in one response at the end (simpler first pass)?

Anthropic call timing — per-check as images stream in, or batched after the whole scan finishes?

Image storage — local disk (path in SQL) vs. Azure Blob Storage (fully cloud-backed, extra cost)?

Record locking — editable after save, or immutable once confirmed?

Duplicate detection — flag likely-duplicate checks (same amount + bank + date, or an image hash)?

Connectivity fallback — since Azure SQL needs internet, does the app need offline queuing if connectivity drops mid-batch, or is "show an error and retry" acceptable for personal use?